



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas

PROYECTO

**"APLICACIÓN MÓVIL COLABORATIVA PARA MEJORAR LA
EFECTIVIDAD EN LA ASISTENCIA DE MASCOTAS PERDIDAS
CON PARTICIPACIÓN DE VOLUNTARIOS EN LA CIUDAD DE
TACNA, 2026"**

Curso: Construcción de software II

Docente: RICARDO EDUARDO VALCARCEL ALVARADO

Integrantes:

Chite Quispe Brian Danilo ***(2021070015)***

Anampa Pancca David Jordan ***(2022074268)***

Tacna – Perú

2026

INDICE

1. Introducción	3
2. Objetivo.....	3
Objetivo General.....	3
Objetivos Específicos.....	3
3. Inventario Product Backlog	4
Requerimientos Funcionales	4
Requerimientos No Funcionales	6
4. Planificación de Sprints con Fecha	6
Sprint 1 — Módulo de Seguridad y Cuenta de Usuario.....	6
Sprint 2 — Módulo de Reportes e Inteligencia Artificial	7
Sprint 3 — Módulo de Visualización y Feed.....	7
Sprint 4 — Módulo de Comunicación y Notificaciones	8
Sprint 5 — Módulo de Comunidad.....	8
Sprint 6 — Módulo Administrativo	8
Sprint 7 — Módulo de Analítica y Estadísticas	9
Sprint 8 — Aseguramiento de Calidad y Auditoría de Código.....	9
5. Descripción Técnica de cada Sprint.....	10
Sprint 1 — Módulo de Seguridad y Cuenta.....	10
Sprint 2 — Módulo de Reportes e Inteligencia Artificial	10
Sprint 3 — Módulo de Visualización y Feed.....	11
Sprint 4 — Módulo de Comunicación y Notificaciones	12
Sprint 5 — Módulo de Comunidad.....	12
Sprint 6 — Módulo Administrativo	13
Sprint 7 — Módulo de Analítica.....	13
Sprint 8 — Aseguramiento de Calidad	14
6. Criterios de Aceptación Generales.....	14
7. Definición de Terminado (Definition of Done)	15

1. Introducción

Actualmente, la pérdida de mascotas representa un problema frecuente en la ciudad de Tacna, afectando tanto a dueños como a voluntarios y organizaciones de apoyo animal. Muchas veces la difusión de información se realiza mediante redes sociales de manera desorganizada, dificultando la localización rápida de las mascotas y reduciendo las posibilidades de recuperación.

Frente a esta problemática, el presente proyecto propone el desarrollo de una aplicación móvil colaborativa denominada "**SOS Mascota**" (`sos_mascotas`), orientada a facilitar el registro y seguimiento de mascotas perdidas, encontradas o en adopción. La aplicación integra funcionalidades como publicación de reportes, visualización geográfica mediante mapas interactivos con OpenStreetMap, registro de avistamientos con geolocalización GPS, notificaciones automáticas mediante Firebase Cloud Messaging, algoritmos de coincidencia biométrica con TensorFlow Lite, y herramientas de administración y moderación comunitaria.

El proyecto es desarrollado en **Flutter** con backend en **Firebase** (Authentication, Firestore, Storage, Messaging), aplicando una metodología de trabajo incremental basada en **Sprints SCRUM**, permitiendo realizar entregas progresivas y funcionales del sistema durante el periodo de desarrollo del curso de Construcción de Software II.

El presente documento contiene la planificación general del proyecto, los requerimientos identificados, el inventario funcional del sistema y la organización de actividades mediante Sprint Backlog para el seguimiento del desarrollo de la aplicación móvil.

2. Objetivo

Objetivo General

Desarrollar una aplicación móvil colaborativa que permita mejorar la efectividad en la asistencia y localización de mascotas perdidas mediante la participación de usuarios y voluntarios en la ciudad de Tacna, utilizando tecnologías Flutter, Firebase y TensorFlow Lite bajo una metodología incremental SCRUM.

Objetivos Específicos

- Permitir a los usuarios registrar reportes de mascotas perdidas con fotografías validadas por inteligencia artificial local.

- Facilitar la visualización de reportes mediante mapas interactivos georreferenciados de Tacna y listados públicos con filtros.
- Implementar un algoritmo de coincidencia inteligente basado en similitud coseno de embeddings TFLite para detectar coincidencias automáticas entre reportes y avistamientos.
- Implementar un sistema de notificaciones push y comunicación directa entre usuarios para agilizar la recuperación de mascotas.
- Desarrollar herramientas administrativas para la gestión de usuarios, reportes y estadísticas dentro del sistema.
- Gamificar la participación comunitaria mediante el sistema de recompensas PataCoins.
- Garantizar un sistema con tiempos de respuesta adecuados, disponibilidad y facilidad de uso para usuarios sin experiencia técnica en Tacna.
- Aplicar una metodología incremental basada en Sprints para controlar y supervisar el avance del proyecto.

3. Inventario Product Backlog

Requerimientos Funcionales

N° PBI	Product Backlog Item	RF
1	Autenticación de usuario (login con correo y contraseña) con verificación de rol en Firestore	RF-001
2	Registro de nuevo usuario con validación de DNI mediante API RENIEC	RF-002
3	Recuperación de contraseña por correo electrónico mediante Firebase Auth	RF-003
4	Verificación de cuenta mediante correo electrónico antes de acceder al sistema	RF-004
5	Registro de reporte de mascota perdida (Formulario Wizard en 3 Pasos)	RF-005

N° PBI	Product Backlog Item	RF
6	Validación y clasificación de imágenes con IA local (TensorFlow Lite MobileNet)	RF-006
7	Visualización y filtrado de reportes de mascotas (Feed principal con chips de filtro)	RF-007
8	Registro de avistamiento de mascota con geolocalización GPS	RF-008
9	Algoritmo de coincidencia inteligente y similitud coseno de embeddings	RF-009
10	Visualización de mascotas en mapa interactivo (OpenStreetMap / FlutterMap)	RF-010
11	Sistema de notificaciones push y alertas comunitarias (FCM HTTP v1)	RF-011
12	Sistema de comentarios, reacciones y respuestas en hilos comunitarios	RF-012
13	Mensajería privada y chat en tiempo real entre voluntario y dueño	RF-013
14	Generación y exportación de cartel de búsqueda en formato PDF	RF-014
15	Gestión de perfil de usuario y monedero de recompensas (PataCoins)	RF-015
16	Panel de administración, moderación de usuarios y control de roles	RF-016
17	Dashboard estadístico y métricas generales del sistema para administradores	RF-017

Fuente: Elaboración propia del equipo de trabajo.

Requerimientos No Funcionales

N° PBI	Product Backlog Item	RNF
18	Rendimiento — Tiempos de respuesta inferiores a 2 segundos para operaciones en Firestore	RNF-001
19	Disponibilidad — SLA de 99.5% garantizado por la infraestructura Firebase de Google Cloud	RNF-002
20	Usabilidad — Flujo de avistamiento completable en menos de 15 segundos por un voluntario en la vía pública	RNF-003
21	Seguridad — Validación de identidad obligatoria mediante DNI RENIEC para todos los usuarios	RNF-004
22	Escalabilidad — Soporte para hasta 1,000 usuarios concurrentes en Tacna sin degradación visible	RNF-005

Fuente: Elaboración propia del equipo de trabajo.

4. Planificación de Sprints con Fecha

El proyecto se organiza en ocho Sprints semanales consecutivos, del 11/05/2026 al 05/07/2026.

Sprint 1 — Módulo de Seguridad y Cuenta de Usuario

Item del Backlog	Requerimiento	Fecha de Inicio	Fecha de Fin
RF-001: Login de usuarios con Firebase Auth y verificación de rol	RF-001	11/05/2026	17/05/2026
RF-002: Registro de usuarios con validación DNI RENIEC	RF-002	11/05/2026	17/05/2026
RF-004: Verificación de cuenta mediante correo electrónico	RF-004	11/05/2026	17/05/2026
RF-015 (Parcial): Gestión de perfil de	RF-015	11/05/2026	17/05/2026

Item del Backlog	Requerimiento	Fecha de Inicio	Fecha de Fin
usuario (datos básicos)			

Duración: 7 días — Equipo: Backend / Auth Lead

Sprint 2 — Módulo de Reportes e Inteligencia Artificial

Item del Backlog	Requerimiento	Fecha de Inicio	Fecha de Fin
RF-005: Formulario Wizard de 3 pasos para reportar mascota perdida	RF-005	18/05/2026	24/05/2026
RF-006: Validación y clasificación de imágenes con TFLite MobileNet	RF-006	18/05/2026	24/05/2026
RF-008: Registro de avistamiento con cámara y coordenadas GPS	RF-008	18/05/2026	24/05/2026
RF-010: Mapa interactivo con FlutterMap y marcadores OpenStreetMap	RF-010	18/05/2026	24/05/2026

Duración: 7 días — Equipo: AI / Mobile Engineer + Frontend Lead

Sprint 3 — Módulo de Visualización y Feed

Item del Backlog	Requerimiento	Fecha de Inicio	Fecha de Fin
RF-007: Feed de reportes con filtros por especie, distrito y estado	RF-007	25/05/2026	31/05/2026
RF-009: Algoritmo de coincidencia coseno de embeddings	RF-009	25/05/2026	31/05/2026
RF-014: Generación y exportación de cartel PDF de búsqueda	RF-014	25/05/2026	31/05/2026
RF-015 (Completar): Historial de	RF-015	25/05/2026	31/05/2026

Item del Backlog	Requerimiento	Fecha de Inicio	Fecha de Fin
PataCoins y transacciones			

Duración: 7 días — Equipo: Frontend Lead + AI Engineer

Sprint 4 — Módulo de Comunicación y Notificaciones

Item del Backlog	Requerimiento	Fecha de Inicio	Fecha de Fin
RF-011: Notificaciones push con Firebase Cloud Messaging v1	RF-011	01/06/2026	07/06/2026
RF-013: Chat en tiempo real entre voluntario y dueño de mascota	RF-013	01/06/2026	07/06/2026

Duración: 7 días — Equipo: Backend Lead

Sprint 5 — Módulo de Comunidad

Item del Backlog	Requerimiento	Fecha de Inicio	Fecha de Fin
RF-012: Comentarios, reacciones y respuestas en hilos comunitarios	RF-012	08/06/2026	14/06/2026
RF-003: Recuperación de contraseña por correo electrónico	RF-003	08/06/2026	14/06/2026

Duración: 7 días — Equipo: Frontend / Mobile Developer

Sprint 6 — Módulo Administrativo

Item del Backlog	Requerimiento	Fecha de Inicio	Fecha de Fin
RF-016: Panel administrativo — Moderación de usuarios y cambio de roles	RF-016	15/06/2026	21/06/2026
RF-016 (Parcial): Gestión del perfil del administrador	RF-016	15/06/2026	21/06/2026

Duración: 7 días — Equipo: Fullstack Developer

Sprint 7 — Módulo de Analítica y Estadísticas

Item del Backlog	Requerimiento	Fecha de Inicio	Fecha de Fin
RF-017: Dashboard estadístico con gráficos por especie y distrito	RF-017	22/06/2026	28/06/2026
RF-016 (Completar): Gestión y moderación de reportes por administrador	RF-016	22/06/2026	28/06/2026

Duración: 7 días — Equipo: Fullstack Developer

Sprint 8 — Aseguramiento de Calidad y Auditoría de Código

Item del Backlog	Actividad	Fecha de Inicio	Fecha de Fin
Pruebas unitarias de módulos críticos	Módulos Auth, Reportes, Avistamientos	29/06/2026	05/07/2026
Auditoría de código (9 Reglas de Código Limpio)	Laboratorio 01	29/06/2026	05/07/2026
Refactorización de hallazgos críticos	39 hallazgos detectados	29/06/2026	05/07/2026
Pruebas de rendimiento, usabilidad y disponibilidad (RNF-001 a RNF-005)	Pruebas no funcionales	29/06/2026	05/07/2026
Corrección de errores y bugs finales	QA General	29/06/2026	05/07/2026

Duración: 7 días — Equipo: QA Engineer + Auditor Senior

5. Descripción Técnica de cada Sprint

Sprint 1 — Módulo de Seguridad y Cuenta

Objetivo: Implementar la autenticación segura, el registro con validación de identidad RENIEC y la persistencia de sesión.

Archivos Clave Desarrollados:

- `lib/vista/auth/pantalla_login.dart` — Interfaz de inicio de sesión con validación de formulario.
- `lib/vistamodelo/auth/login_vm.dart` — Lógica de autenticación, verificación de rol y actualización de token FCM.
- `lib/vista/auth/pantalla_registro.dart` — Formulario de alta con consulta de DNI.
- `lib/vistamodelo/auth/registro_vm.dart` — Lógica de registro, consulta API RENIEC y creación en Firebase.
- `lib/servicios/auth_servicio.dart` — Servicio de autenticación con Firebase Auth.
- `lib/servicios/api_dni_servicio.dart` — Servicio de consulta REST a la API RENIEC (miapi.cloud).
- `lib/utills/auth_wrapper.dart` — Stream reactivo para verificación continua de estado de cuenta.

Criterios de Completado:

- Login exitoso con correo y contraseña redirige al feed principal o panel administrativo según el rol.
- Registro con DNI válido auto-completa nombres y apellidos desde RENIEC.
- El correo de verificación se envía automáticamente tras el registro.
- Cuentas con `activo == false` son expulsadas en tiempo real.

Sprint 2 — Módulo de Reportes e Inteligencia Artificial

Objetivo: Implementar el wizard de reporte de mascotas, el clasificador TFLite y el registro de avistamientos con GPS.

Archivos Clave Desarrollados:

- `lib/vista/reportes/pantalla_reporte_mascota.dart` — Wizard de 3 pasos (datos, ubicación, recompensa).
- `lib/vistamodelo/reportes/reportes_vm.dart` — Lógica de guardado en Firestore y despacho de notificación push global.
- `lib/vista/reportes/pantalla_avistamiento.dart` — Captura rápida con cámara y GPS.
- `lib/vistamodelo/reportes/avistamiento_vm.dart` — Lógica de avistamiento con acreditación de PataCoins.
- `lib/servicios/servicio_tflite.dart` — Clasificación MobileNet y extracción de embeddings de 1280 dimensiones.
- `lib/utills/imagen_util.dart` — Centralización de validación y subida de imágenes (Regla DRY).
- `lib/vista/mapa/pantalla_mapa_interactivo.dart` — Mapa FlutterMap con marcadores azules y naranjas.

Criterios de Completado:

- Las imágenes con confianza TFLite $< 60\%$ son rechazadas con mensaje informativo.
- El reporte se guarda con latitud, longitud, URL de imagen y vector de embeddings.
- El avistamiento acredita 10 PataCoins al voluntario de forma automática.

Sprint 3 — Módulo de Visualización y Feed

Objetivo: Implementar el feed reactivo de reportes con filtros, el algoritmo de similitud coseno y la generación de afiches PDF.

Archivos Clave Desarrollados:

- `lib/vista/reportes/pantalla_ver_reportes.dart` — Feed con chips de filtro por especie y distrito.
- `lib/vista/reportes/pantalla_mis_reportes.dart` — Gestión de reportes propios del usuario.
- Algoritmo de similitud coseno integrado en `avistamiento_vm.dart` con radio de 9.0 km.
- Motor de generación PDF con paquetes `pdf` y `printing`.
-

Criterios de Completado:

- El feed se actualiza en tiempo real sin necesidad de recargar la pantalla.
- El algoritmo de cotejo ejecuta en menos de 3 segundos para hasta 50 reportes activos.
- El afiche PDF incluye foto, datos, mapa de referencia y contacto del dueño.

Sprint 4 — Módulo de Comunicación y Notificaciones

Objetivo: Implementar el sistema de notificaciones push FCM y el chat en tiempo real.

Archivos Clave Desarrollados:

- `lib/servicios/notificacion_servicio.dart` — Despacho de notificaciones push mediante FCM HTTP v1.
- `lib/vista/usuario/pantalla_notificacion.dart` — Centro de notificaciones con historial.
- `lib/vista/chat/pantalla_chat.dart` — Sala de chat en tiempo real con Firestore streams.

Criterios de Completado:

- Las notificaciones push llegan al dueño cuando se detecta una posible coincidencia.
- Los mensajes del chat se sincronizan en tiempo real entre ambos dispositivos.
- Las notificaciones locales funcionan cuando la aplicación está en segundo plano.

Sprint 5 — Módulo de Comunidad

Objetivo: Implementar los comentarios comunitarios con hilos de respuesta y la recuperación de contraseña.

Archivos Clave Desarrollados:

- `lib/vista/usuario/pantalla_comentarios.dart` — Comentarios reactivos en tiempo real.
- `lib/vista/usuario/replies_page.dart` — Hilos de respuestas anidadas.
- `lib/vistamodelo/comentarios/comentarios_viewmodel.dart` — Lógica de comentarios con `ChangeNotifier`.
- `lib/vista/auth/pantalla_recuperar.dart` — Envío de enlace de restablecimiento por Firebase Auth.

Criterios de Completado:

- Los nuevos comentarios aparecen en tiempo real sin recargar la pantalla.
- Las reacciones de Like/Dislike actualizan atómicamente los contadores en Firestore.
- El correo de recuperación se envía en menos de 60 segundos.

Sprint 6 — Módulo Administrativo

Objetivo: Implementar el panel de administración con moderación de usuarios y control de roles.

Archivos Clave Desarrollados:

- `lib/vista/admin/pantalla_inicio_admin.dart` — Dashboard del administrador.
- `lib/vista/admin/pantalla_adminusuarios.dart` — Lista de usuarios con conmutador de estado y rol.
- `lib/vistamodelo/admin/adminusuario_vm.dart` — Lógica de suspensión y cambio de rol.
- `lib/vista/admin/pantallaAdminReporte.dart` — Moderación de reportes publicados.

Criterios de Completado:

- La suspensión de un usuario activo lo expulsa en tiempo real de la sesión activa.
- El cambio de rol de `usuario` a `admin` es efectivo de inmediato en el siguiente inicio de sesión.

Sprint 7 — Módulo de Analítica

Objetivo: Implementar el dashboard estadístico con gráficos dinámicos para el administrador.

Archivos Clave Desarrollados:

- `lib/vista/admin/PantallaAdminEstadisticas.dart` — Dashboard con gráficos estadísticos.
- `lib/vistamodelo/admin/AdminEstadisticas_vm.dart` — Agregaciones sobre colecciones Firestore.

Criterios de Completado:

- Los gráficos muestran la distribución correcta de mascotas por especie y por distrito de Tacna.

- El indicador de tasa de éxito (mascotas encontradas / total reportadas) se calcula en tiempo real.

Sprint 8 — Aseguramiento de Calidad

Objetivo: Auditar el código fuente, corregir hallazgos críticos y garantizar 100% de pruebas unitarias aprobadas.

Actividades Realizadas:

- Aplicación de las 9 Reglas de Código Limpio (Laboratorio 01 — Auditoría de Código).
- Extracción de `ImagenUtil.validarYSubirFoto()` para eliminar duplicación entre ViewModels.
- Centralización de `_buildNotifData()` en `NotificacionServicio`.
- Encapsulación de token Bearer en `AppConfig` con soporte `--dart-define`.
- Tipado de excepciones `FirebaseAuthException` en `auth_servicio.dart` y `login_vm.dart`.
- Reemplazo de `print()` por `debugPrint()` en todos los ViewModels.
- Encapsulación de `navigatorKey` en `NavigationService`.
- Encapsulación de `flutterLocalNotificationsPlugin` en `LocalNotificationService.instance`.
- Adición de llaves `{}` en todos los bloques `if / else if` de un solo línea.

Resultado Final: `flutter test` — 3/3 casos aprobados (100%). 0 regresiones.

6. Criterios de Aceptación Generales

Todos los Product Backlog Items implementados en cada Sprint deben cumplir los siguientes criterios transversales antes de ser considerados completados:

- El flujo de usuario descrito en la narrativa del caso de uso se ejecuta sin errores de tiempo de ejecución en dispositivo Android 13.
- Los datos se persisten correctamente en Cloud Firestore y se reflejan en tiempo real en otros dispositivos conectados.
- El aplicativo no muestra errores no manejados al usuario (no hay pantallas rojas de error de Flutter en producción).
- La funcionalidad responde en menos de 2 segundos para operaciones que requieren red (RNF-001).

- El código desarrollado supera la revisión de código del equipo sin observaciones críticas pendientes.
- Los campos obligatorios de los formularios tienen validación visible antes del envío.

7. Definición de Terminado (Definition of Done)

Un elemento del backlog se considera **Terminado** cuando cumple todos los siguientes criterios:

Criterio	Verificación
Funcionalidad implementada y verificada en emulador y dispositivo físico	Revisión manual por QA Engineer
Código formateado con <code>dart format</code> .	Verificado con CI pre-commit
Prueba unitaria agregada o actualizada para la lógica modificada	<code>flutter test</code> reporta 0 fallos
Reglas de código limpio aplicadas (DRY, Fail Fast, sin magic numbers, etc.)	Code review aprobado
Documentación DartDoc de métodos públicos actualizada	Revisión de comentarios en código
Captura de pantalla de evidencia tomada para el informe de laboratorio	Adjunta en carpeta <code>evidencias/</code>
Merge realizado a la rama <code>UNIDAD-I</code> del repositorio GitHub	Pull request cerrado